OASIS INFOBYTE

DATA ANALYTICS INTERNSHIP

LEVEL 1 – TASK 4

SENTIMENT ANALYSIS

Final Project Report

Submitted By

Madala Sai Sprisha

Internship

OASIS INFOBYTE – Data Analytics Internship

Domain

Data Analytics

Project Title

Level 1 – Task 4: Sentiment Analysis

Technologies Used

Python • Pandas • NumPy • NLTK • Scikit-learn • Matplotlib • Seaborn • WordCloud • Jupyter Notebook

Modules Implemented

Data Inspection • Text Preprocessing • TF-IDF Feature Extraction • Naive Bayes • Logistic Regression • Model Evaluation • Confusion Matrix • WordCloud • Error Analysis

Submitted To

OASIS INFOBYTE

Year

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to OASIS INFOBYTE for providing me with the opportunity to complete Level 1 – Task 4, Sentiment Analysis. This project provided practical exposure to natural language processing, text preprocessing, feature extraction, supervised machine learning, model evaluation, and visualization.

I also thank the mentors and coordinators for providing the internship task requirements and learning resources. The structured workflow helped me understand how unstructured social-media text can be transformed into numerical features and used for automated sentiment classification.

The implementation was completed in a Jupyter Notebook using Python and the provided Twitter training dataset. The project compares Multinomial Naive Bayes and Logistic Regression and documents the complete workflow from data inspection to error analysis and final model selection.

Finally, I appreciate everyone who directly or indirectly supported the successful completion of this project. The experience strengthened my practical understanding of text analytics and machine-learning based classification.

ABSTRACT

This project focuses on Sentiment Analysis of Twitter text using Python and machine learning. The objective is to classify text into three sentiment categories: Positive, Negative, and Neutral. The workflow begins with dataset inspection and quality assessment, followed by removal of missing and duplicate records, selection of the required sentiment classes, text preprocessing, TF-IDF feature extraction, stratified train-test splitting, supervised model training, evaluation, visualization, and error analysis.

The original dataset contains 74,682 records and 4 columns: ID, Entity, Sentiment, and Text. Initial inspection identified 686 missing Text values and 2,700 duplicate records. The source contained four sentiment labels, including Irrelevant; because the internship task requires Positive, Negative, and Neutral classification, Irrelevant records were excluded. After cleaning and filtering, 59,119 records remained, and text preprocessing produced 57,606 usable text samples.

The cleaned text was transformed into 5,000 TF-IDF features using unigram and bigram representations. An 80:20 stratified split produced 46,084 training samples and 11,522 testing samples. Multinomial Naive Bayes achieved an accuracy of 71.16%, while Logistic Regression achieved 75.73%. Logistic Regression was therefore selected as the best-performing model based on the highest F1-score of 0.7560.

The project also generated sentiment distribution charts, model comparison visuals, confusion matrices, sentiment-specific WordClouds, and an error-analysis table containing five representative misclassified examples. The results demonstrate that machine learning can provide useful automated sentiment classification, while ambiguity, sarcasm, short text, and contextual language remain important challenges.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Sentiment Analysis project focuses on transforming unstructured Twitter text into meaningful sentiment categories using natural language processing and supervised machine learning. The implementation follows a reproducible pipeline: dataset loading, data-quality assessment, sentiment filtering, text cleaning, feature extraction, model training, evaluation, visualization, and error analysis.

The project uses the Twitter training dataset supplied for the internship. Since the task requires three sentiment classes, the Irrelevant class is excluded before modelling. The cleaned text is represented numerically through TF-IDF and then supplied to two classification algorithms: Multinomial Naive Bayes and Logistic Regression.

1.2 Problem Statement

Large volumes of social-media text are difficult to analyse manually because individual messages are short, informal, context-dependent, and often contain spelling variations, slang, or ambiguous expressions. A sentiment-analysis system can reduce this manual effort by automatically assigning sentiment labels to text and providing an overall view of user opinion.

The problem addressed in this project is therefore to build and evaluate machine-learning models capable of classifying Twitter text into Positive, Negative, and Neutral categories using a consistent preprocessing and feature-extraction pipeline.

1.3 Project Objectives

1. Load and inspect the Twitter training dataset.
2. Identify missing values, duplicate records, and available sentiment classes.
3. Remove records that do not satisfy the three-class task requirement.
4. Clean and normalize text by removing URLs, mentions, punctuation, stopwords, and very short tokens.
5. Convert cleaned text into numerical TF-IDF features using unigrams and bigrams.
6. Split the data into stratified training and testing subsets.
7. Train Multinomial Naive Bayes and Logistic Regression classifiers.
8. Evaluate the models using accuracy, precision, recall, F1-score, classification reports, and confusion matrices.
9. Generate sentiment distribution and WordCloud visualizations.

10. Perform error analysis and select the best-performing model based on F1-score.

CHAPTER 2: DATASET & TOOLS

2.1 Dataset Description

The project uses the Twitter training dataset stored as twitter_training.csv. The original dataset contains 74,682 records and four fields: ID, Entity, Sentiment, and Text. The ID field identifies the record, Entity represents the associated topic or entity, Sentiment contains the original class label, and Text contains the social-media message to be analysed.

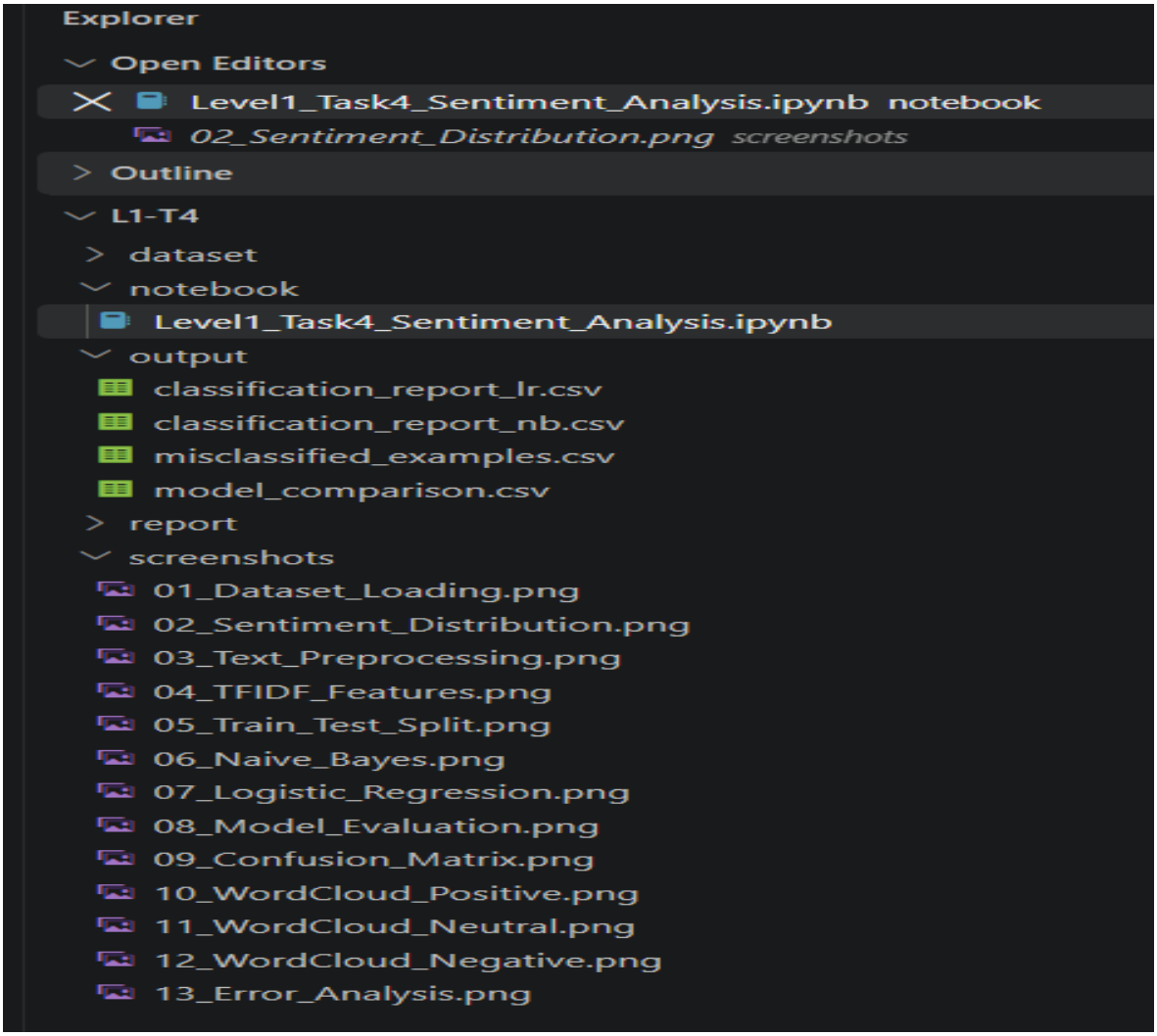
Initial inspection identified 686 missing Text values and 2,700 duplicate rows. The original sentiment distribution contained four categories: Negative (22,542), Positive (20,832), Neutral (18,318), and Irrelevant (12,990). After removal of missing/duplicate records and exclusion of Irrelevant records, 59,119 records remained for the three-class task.

2.2 Tools & Technologies Used

Tool / Technology	Purpose
Python	Programming language for the complete analytics workflow
Pandas	Data loading, cleaning, filtering and tabular analysis
NumPy	Numerical operations and data support
NLTK	Tokenization and stopword processing
Scikit-learn	TF-IDF, train-test split, Naive Bayes, Logistic Regression and evaluation metrics
Matplotlib	Charts and model-performance visualization
Seaborn	Confusion matrices and statistical visualization
WordCloud	Sentiment-specific word-frequency visualization
Jupyter Notebook	Interactive implementation and execution environment

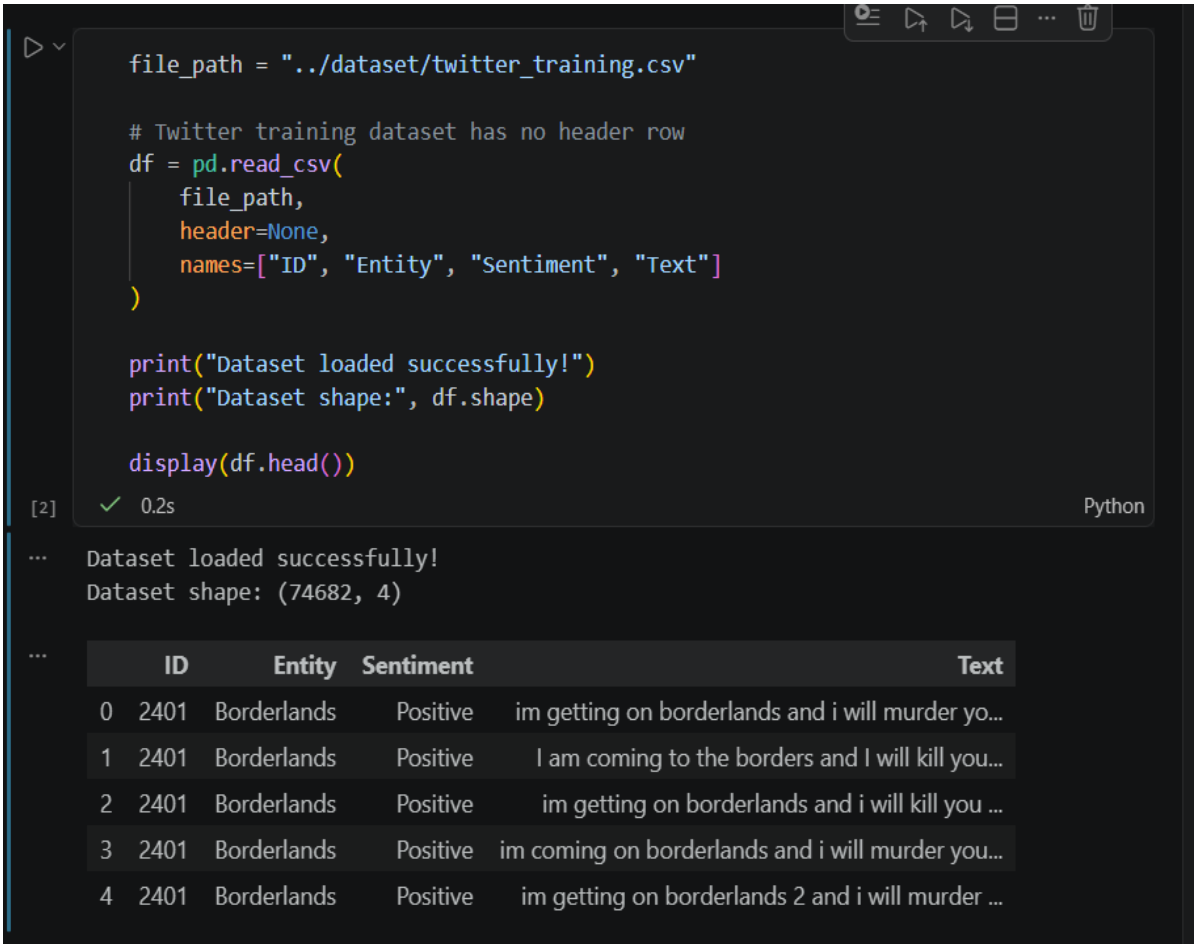
2.3 Project Folder Structure

The project was organized into separate folders for the dataset, notebook implementation, generated outputs, documentation, and screenshot evidence. This structure supports reproducibility and makes the final submission easy to verify.



2.4 Input Dataset

The notebook loads the dataset using the relative path ../dataset/twitter_training.csv. The dataset is read without assuming a header row and is assigned the columns ID, Entity, Sentiment, and Text. The resulting DataFrame has 74,682 rows and 4 columns, confirming successful ingestion.



```
file_path = "../dataset/twitter_training.csv"

# Twitter training dataset has no header row
df = pd.read_csv(
    file_path,
    header=None,
    names=["ID", "Entity", "Sentiment", "Text"]
)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

display(df.head())
```

[2] ✓ 0.2s Python

```
Dataset loaded successfully!
Dataset shape: (74682, 4)
```

	ID	Entity	Sentiment	Text
0	2401	Borderlands	Positive	im getting on borderlands and i will murder yo...
1	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
2	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
3	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
4	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...

Figure 2.4: Dataset Loading and Initial Records

Interpretation:

The screenshot confirms successful loading of the Twitter dataset with 74,682 records and four fields. The first records demonstrate the expected ID, entity, sentiment, and text structure.

CHAPTER 3: SENTIMENT ANALYSIS IMPLEMENTATION

3.1 Library Import & Dataset Loading

The implementation begins by importing Pandas, NumPy, regular-expression utilities, NLTK, Matplotlib, Seaborn, Scikit-learn components, and WordCloud. NLTK resources for English stopwords and tokenization are downloaded before preprocessing. The output and screenshot directories are also created programmatically to keep the workflow organized.

The dataset is loaded with pandas.read_csv using header=None because the supplied CSV does not contain a header row. The columns are explicitly named ID, Entity, Sentiment, and Text. This ensures consistent field names throughout the notebook.

```
file_path = "../dataset/twitter_training.csv"

# Twitter training dataset has no header row
df = pd.read_csv(
    file_path,
    header=None,
    names=["ID", "Entity", "Sentiment", "Text"]
)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

display(df.head())
```

[2] ✓ 0.2s Python

```
Dataset loaded successfully!
Dataset shape: (74682, 4)
```

	ID	Entity	Sentiment	Text
0	2401	Borderlands	Positive	im getting on borderlands and i will murder yo...
1	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
2	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
3	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
4	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...

Figure 3.1: Successful Dataset Loading

Interpretation:

The loading stage establishes a reproducible input structure and confirms that the source contains 74,682 records and four columns.

3.2 Data Inspection & Quality Assessment

The dataset was inspected using column names, data types, missing-value counts, duplicate counts, and sentiment frequencies. The inspection showed that Text contained 686 missing values, while the other columns were complete. A total of 2,700 duplicate rows were detected. The source also contained four sentiment categories: Negative, Positive, Neutral, and Irrelevant.

This inspection was important because model quality depends directly on the consistency and completeness of the training data. Missing text cannot contribute meaningful linguistic features, while duplicate records can bias the learning process by repeatedly presenting identical examples.

3.3 Sentiment Filtering

The internship task requires a three-class classifier covering Positive, Negative, and Neutral sentiment. Therefore, after missing-value and duplicate handling, the Irrelevant category was excluded from the modelling dataset. The filtered dataset contained 59,119 records: 21,698 Negative, 19,713 Positive, and 17,708 Neutral.

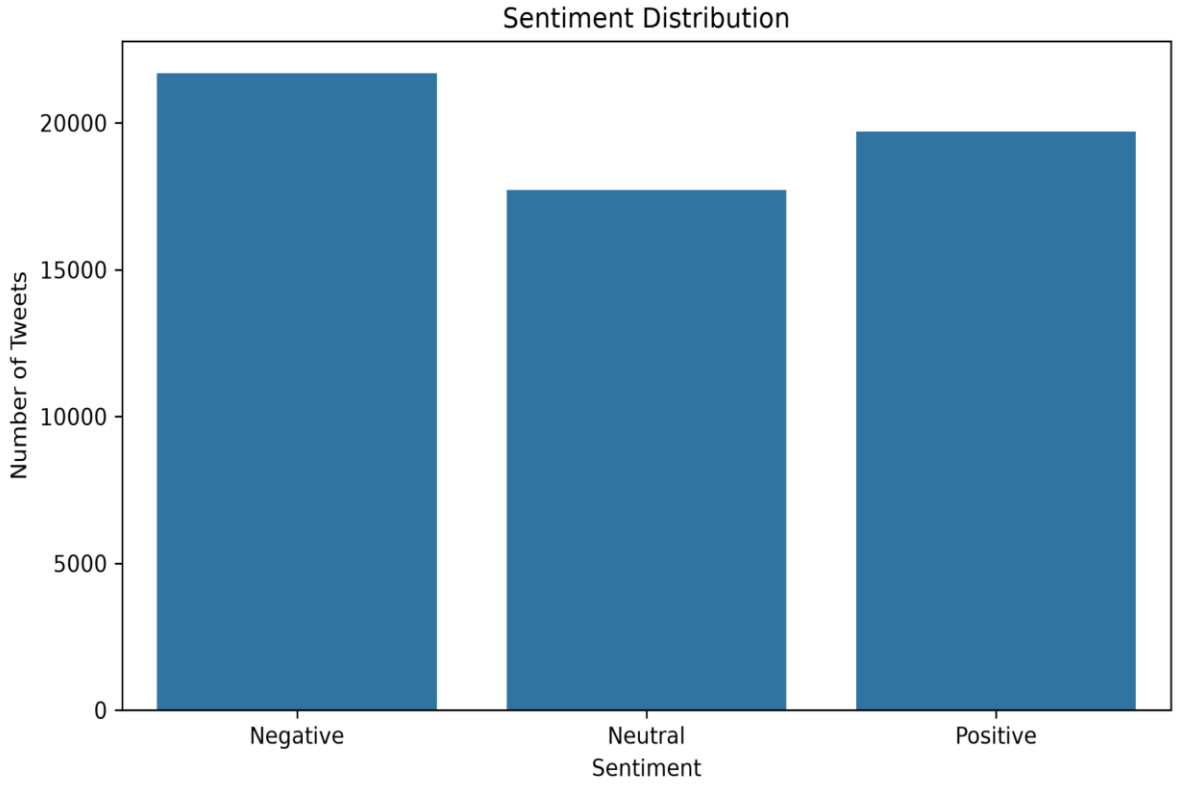


Figure 3.3: Sentiment Distribution After Filtering

Interpretation:

Negative is the largest class at 36.70%, followed by Positive at 33.34% and Neutral at 29.95%. The class proportions are reasonably balanced for a three-class classification task, although Negative has a modestly larger share.

3.4 Text Preprocessing

Text preprocessing was implemented through a dedicated function. Each message was converted to lowercase, URLs and user mentions were removed, hashtag symbols were stripped while retaining the associated word, non-alphabetic characters were removed, and the text was tokenized. English stopwords and tokens shorter than three characters were excluded. The remaining tokens were joined back into a cleaned text string.

After preprocessing, records with empty cleaned text were removed. The resulting modelling dataset contained 57,606 usable text samples, with no empty cleaned texts. This step reduces noise and ensures that each training example contributes meaningful lexical information.

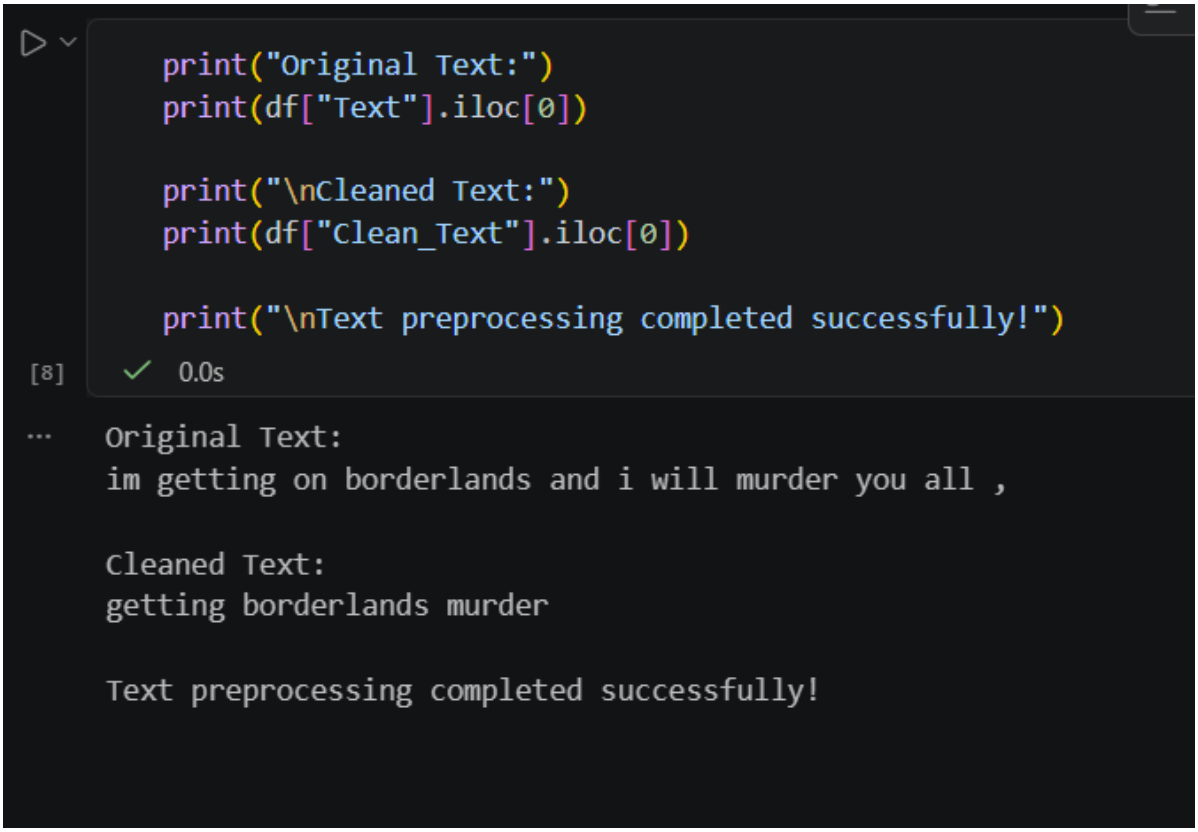


Figure 3.4: Example of Text Preprocessing

Interpretation:

The example shows how an informal Twitter message is reduced to informative tokens. Noise such as punctuation, filler words, and unnecessary wording is removed while the core topic and sentiment-bearing terms are retained.

3.5 Train-Test Split

The cleaned dataset was divided into training and testing subsets using an 80:20 split with random_state=42. Stratification by sentiment was applied so that the class proportions were preserved in both subsets. The split produced 46,084 training samples and 11,522 testing samples.

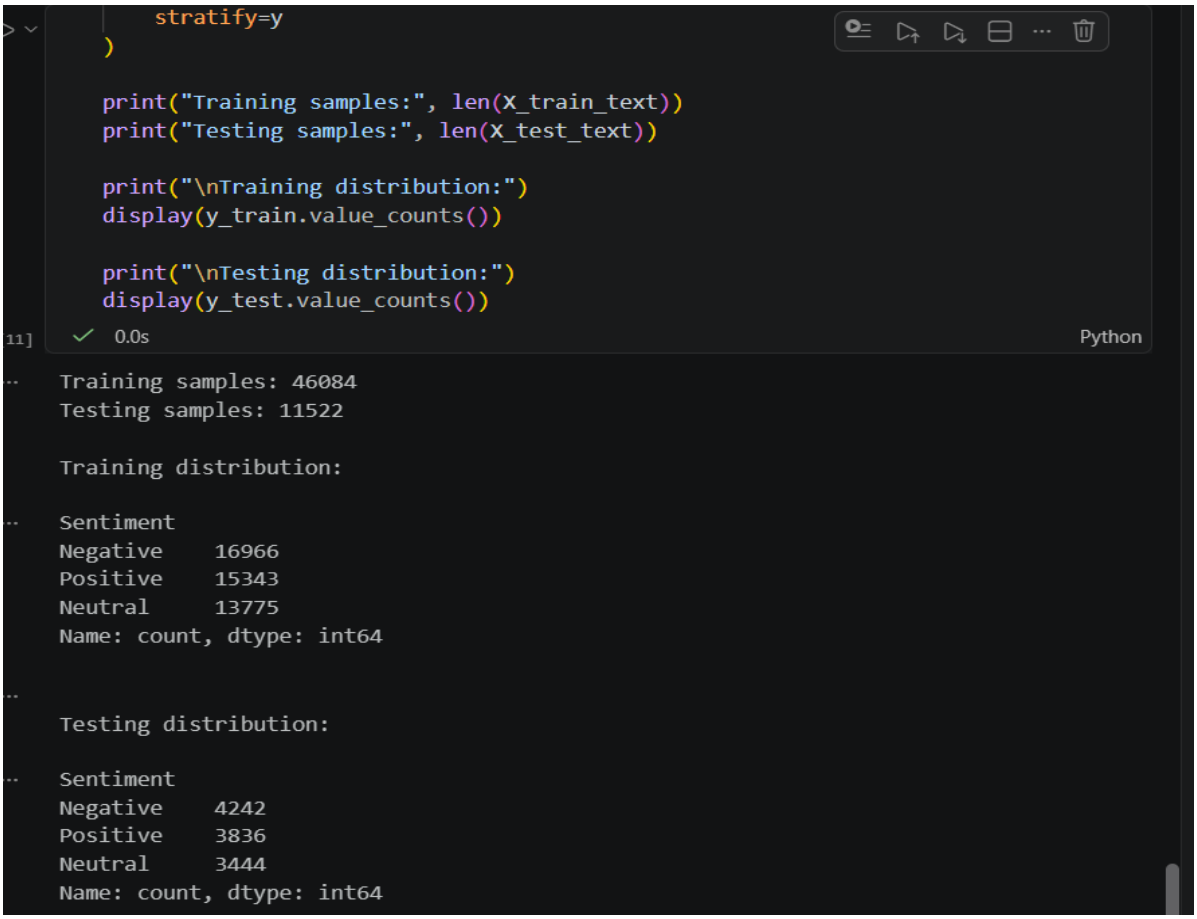


Figure 5: Stratified Train-Test Split

Interpretation:

The training set contains 46,084 samples and the test set contains 11,522 samples. Stratification maintains comparable Negative, Neutral, and Positive distributions across both sets, supporting fair model evaluation.

3.6 TF-IDF Feature Extraction

Machine-learning classifiers require numerical input, so the cleaned text was converted into TF-IDF features. The TfidfVectorizer was configured with max_features=5000, ngram_range=(1,2), min_df=2, and max_df=0.95. This representation captures both individual words and two-word phrases while limiting extremely rare and overly common terms.

The vectorizer was fitted only on the training text and then used to transform the test text. This avoids information leakage from the test set into the feature-learning stage. The resulting training matrix contains 46,084 samples and 5,000 features, while the test matrix contains 11,522 samples and 5,000 features.

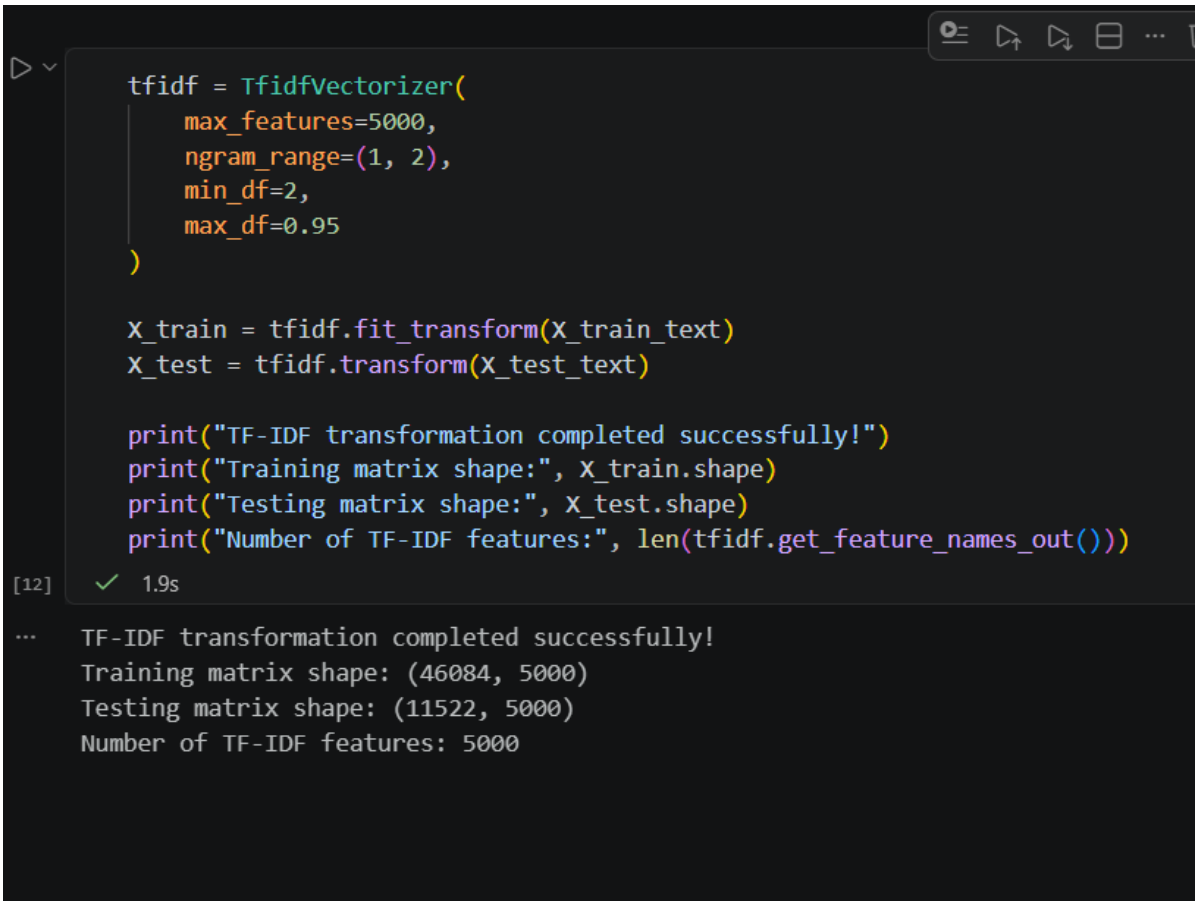


Figure 6: TF-IDF Feature Extraction

Interpretation:

The output confirms successful transformation of the text into a 5,000-feature numerical representation. The consistent feature dimension across training and testing sets makes the data suitable for supervised classification.

3.7 Multinomial Naive Bayes Model

Multinomial Naive Bayes was trained on the TF-IDF representation. The model achieved an accuracy of 0.7116, weighted precision of 0.7145, weighted recall of 0.7116, and weighted F1-score of 0.7067. The model performed best on Negative sentiment, with a recall of 0.81, while Neutral sentiment was more difficult, with a recall of 0.54.

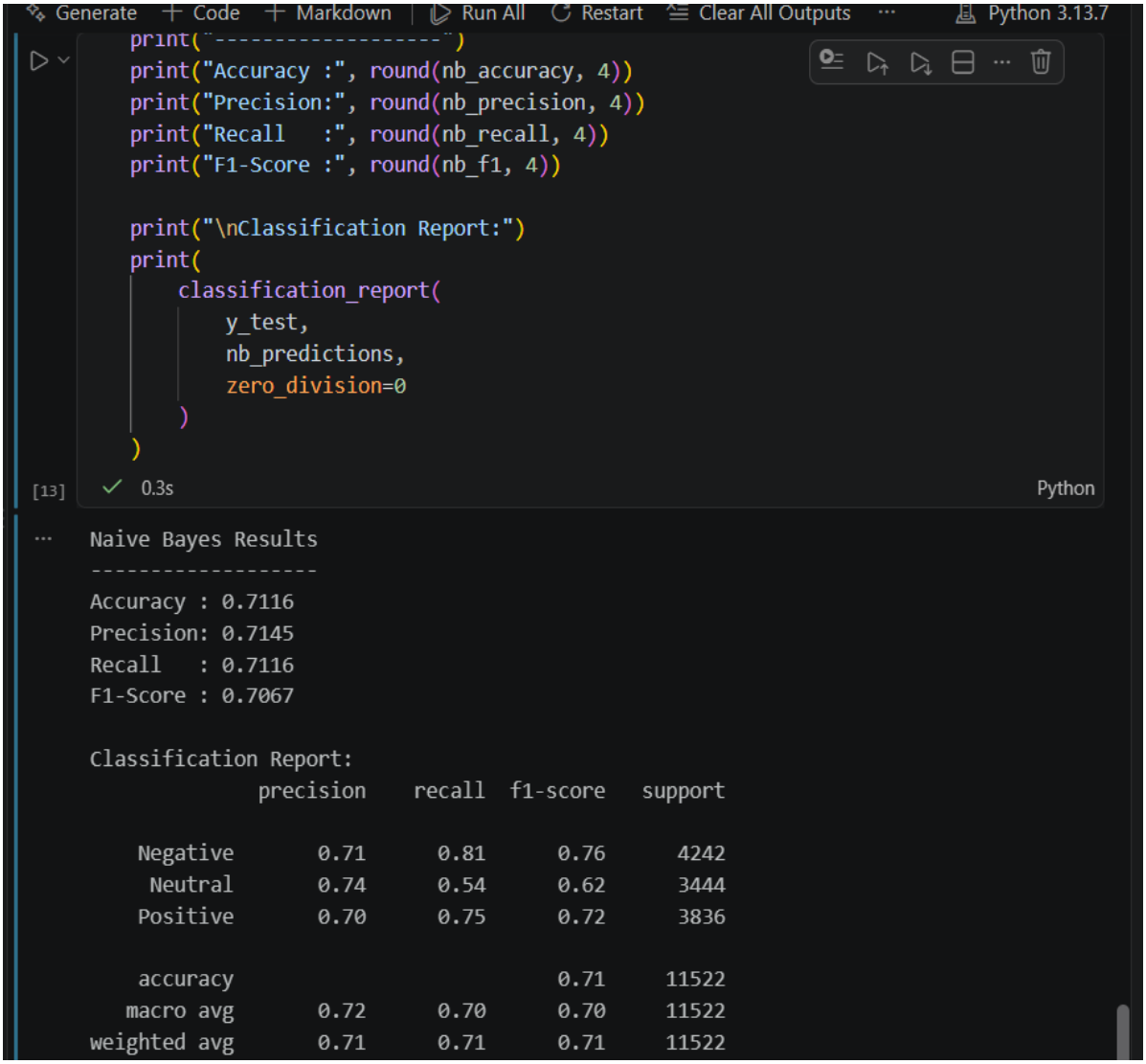


Figure 7: Multinomial Naive Bayes Evaluation

Interpretation:

Naive Bayes provides a solid baseline for text classification, but its lower Neutral recall indicates difficulty separating neutral language from positive or negative expressions when context is limited.

3.8 Logistic Regression Model

Logistic Regression was trained using the same TF-IDF features to provide a directly comparable linear classification model. The classifier was configured with max_iter=1000 and random_state=42. It achieved an accuracy of 0.7573, weighted precision of 0.7568, weighted recall of 0.7573, and weighted F1-score of 0.7560.

The classification report shows improved performance across all three classes compared with Naive Bayes. Negative sentiment achieved a recall of 0.82, Neutral achieved 0.67, and Positive achieved 0.77. This indicates that Logistic Regression captures the TF-IDF feature patterns more effectively for the present dataset.

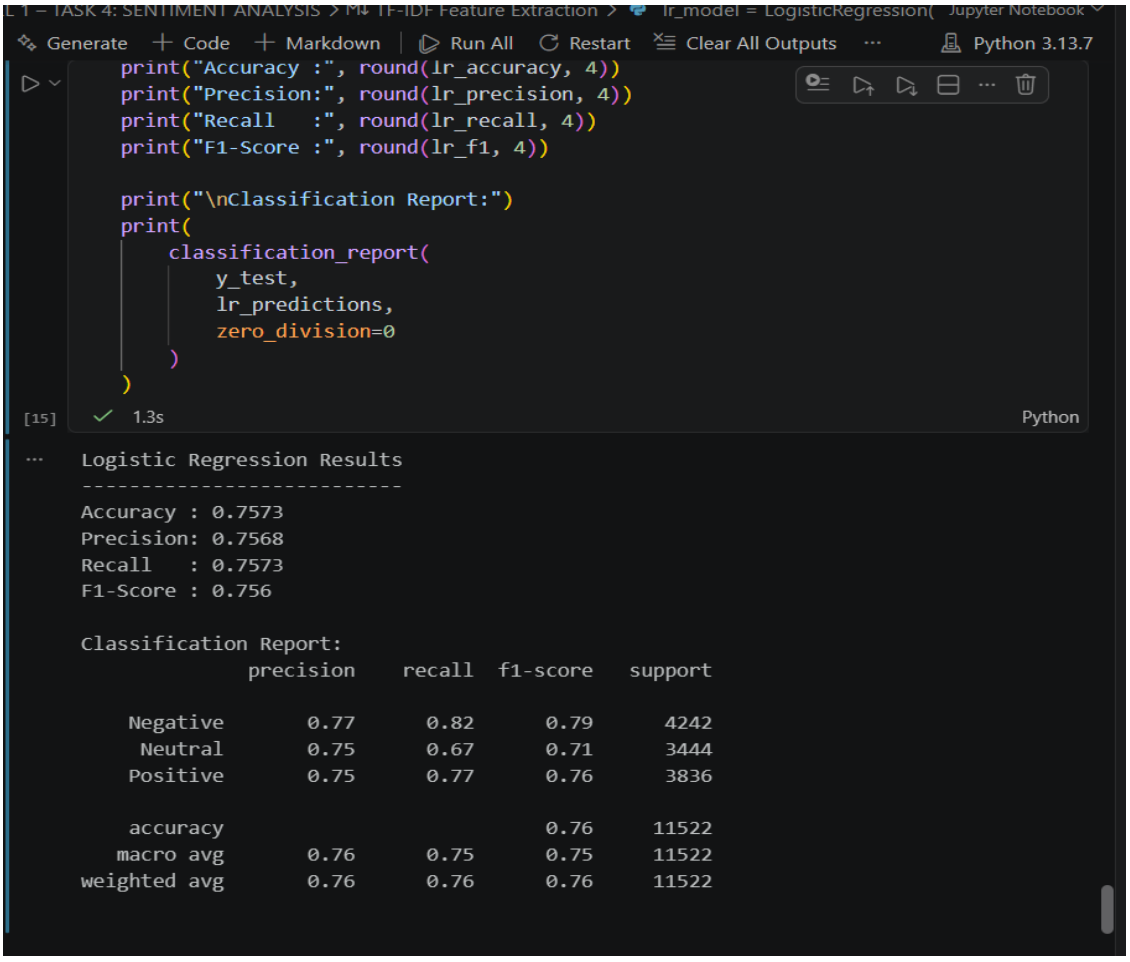


Figure 8: Logistic Regression Evaluation

Interpretation:

Logistic Regression outperforms Naive Bayes on every reported aggregate metric. Its stronger Neutral recall is particularly important because neutral messages are often harder to distinguish from the other classes.

3.9 Model Comparison

The model comparison shows that Logistic Regression performs slightly better than Multinomial Naive Bayes, achieving higher accuracy, precision, recall, and F1-score.

Overall, both models provide strong sentiment-classification performance, with Logistic Regression selected as the better-performing model.

Model	Accuracy	Precision	Recall	F1-Score
Multinomial Naive Bayes	0.7116	0.7145	0.7116	0.7067
Logistic Regression	0.7573	0.7568	0.7573	0.7560

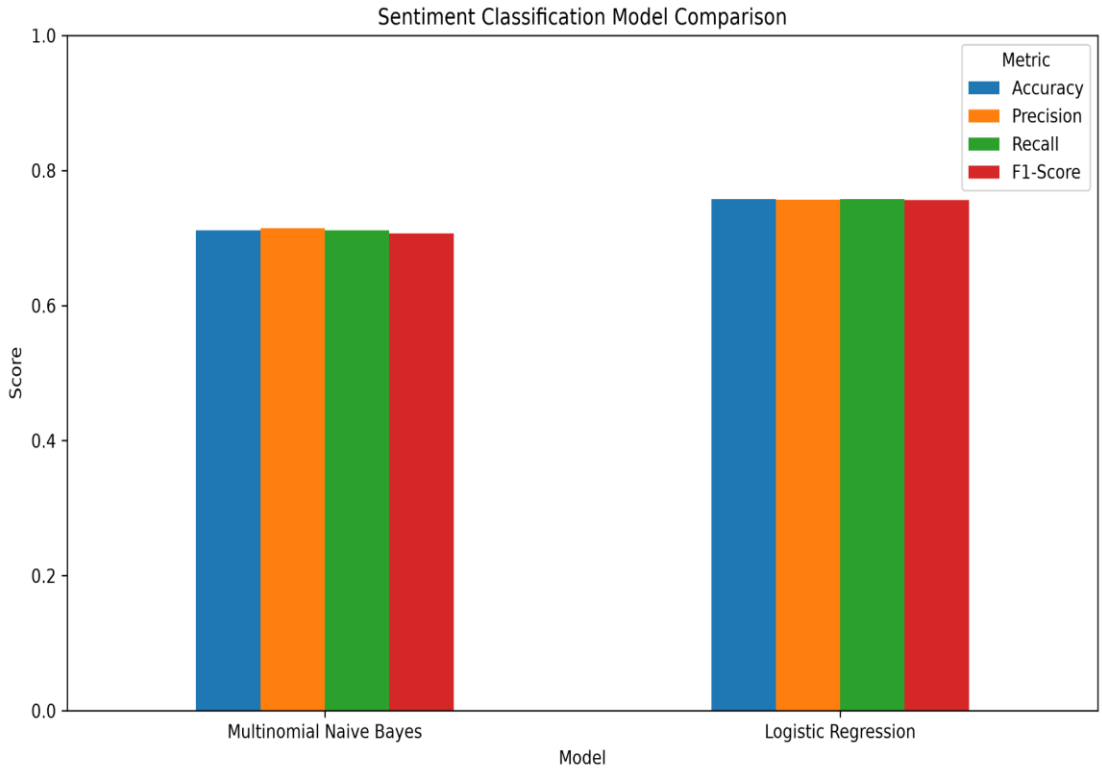


Figure 9: Sentiment Classification Model Comparison

Interpretation:

The bar chart shows that Logistic Regression has higher accuracy, precision, recall, and F1-score than Multinomial Naive Bayes. The consistent improvement across all metrics supports selecting Logistic Regression as the final model.

3.10 Confusion Matrix Analysis

Confusion matrices were generated for both classifiers using the class order Negative, Neutral, and Positive. Logistic Regression correctly classified 3,483 Negative, 2,297 Neutral, and 2,946 Positive test samples. Its largest errors involve cross-class confusion between Negative and Positive, while Neutral also shows some overlap with the other categories.

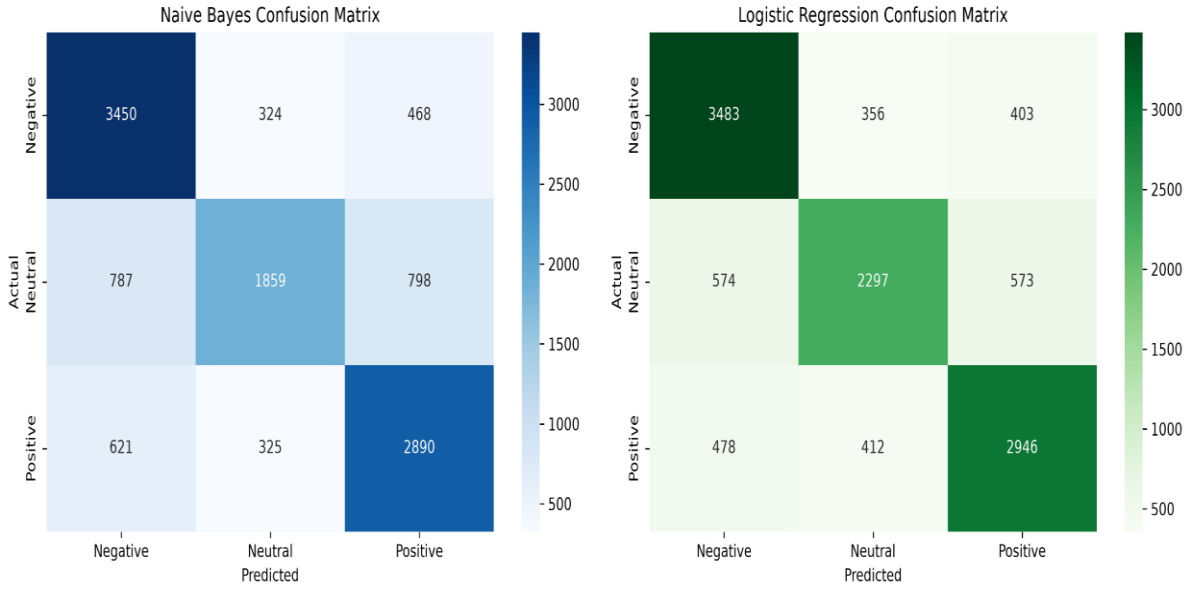


Figure 10: Confusion Matrices for Naive Bayes and Logistic Regression

Interpretation:

The Logistic Regression matrix has stronger diagonal values than the Naive Bayes matrix. The improvement is especially visible for Neutral and Positive predictions, indicating better separation of the three classes.

3.11 WordCloud Analysis

WordClouds were generated separately for Positive, Neutral, and Negative records using the cleaned text. These visualizations provide an intuitive view of frequently occurring terms within each sentiment class. They are exploratory rather than definitive because word frequency alone does not establish sentiment meaning.

Figure 11: Positive Sentiment WordCloud

Interpretation:

The Positive WordCloud contains frequently occurring terms associated with games, entertainment, appreciation, enjoyment, and positive reactions. The visualization gives a quick overview of the vocabulary dominant in positive-labelled text.

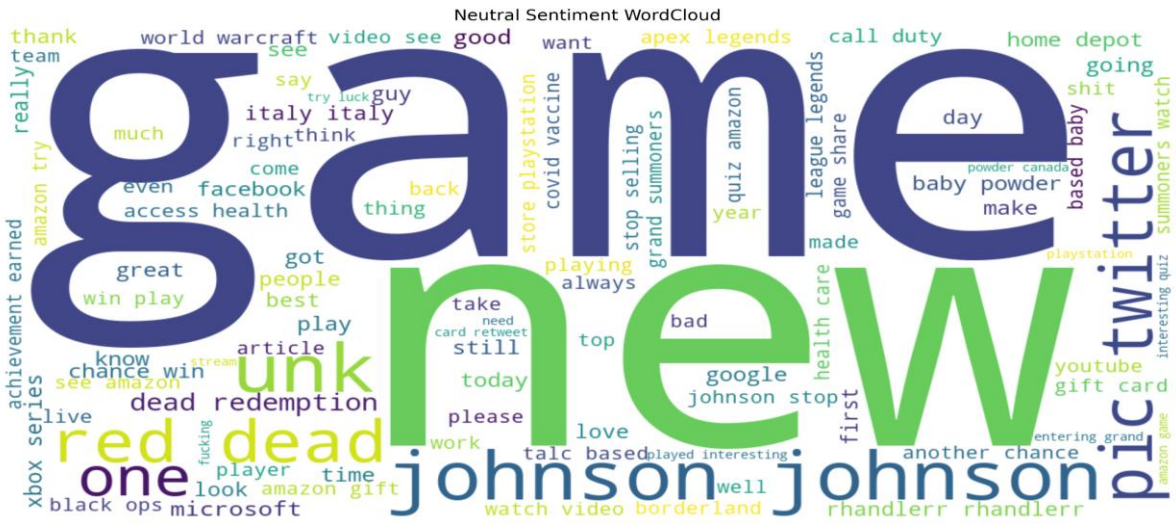


Figure 12: Neutral Sentiment WordCloud

Interpretation:

The Neutral WordCloud contains many topic-specific and informational terms. The vocabulary appears less strongly polarized, which is consistent with the neutral class being more context-dependent.

Figure 13: Negative Sentiment WordCloud

Interpretation:

The Negative WordCloud is dominated by strongly emotional or critical vocabulary. This indicates that negative messages often contain distinctive lexical cues, which helps explain the relatively high Negative recall achieved by both models.

3.12 Error Analysis

Error analysis was performed using Logistic Regression predictions. A total of 2,796 test samples were misclassified. Five representative examples were inspected to understand common causes of classification errors. These included ambiguous wording, sarcasm or informal language, mixed sentiment, short messages with insufficient context, and words whose sentiment depends heavily on surrounding context.

```python
five_errors = misclassified.head(5).copy()

five_errors["Possible Reason"] = [
    "Ambiguous wording or limited context",
    "Sarcasm or informal language",
    "Mixed sentiment within the tweet",
    "Short text with insufficient contextual information",
    "Words may have different sentiment meanings depending on context"
]

display(five_errors)

five_errors.to_csv(
    "../output/misclassified_examples.csv",
    index=False
)
```

[24] ✓ 0.0s Python

	Text	Actual	Predicted	Possible Reason
6	tried	Positive	Negative	Ambiguous wording or limited context
13	microsoft shuts controversial botnet necurs iz...	Neutral	Negative	Sarcasm or informal language
15	best stream months big appreciation sports peo...	Positive	Neutral	Mixed sentiment within the tweet
19	excellent find hold entire parties using zoom ...	Neutral	Positive	Short text with insufficient contextual inform...
22	unfortunately vlad lost body thanks listening	Neutral	Negative	Words may have different sentiment meanings de...

Figure 14: Representative Misclassified Examples

Interpretation:

The error-analysis table demonstrates that even correctly implemented classifiers can struggle with short, ambiguous, sarcastic, or context-dependent text. These errors highlight the limitations of purely lexical TF-IDF representations.

Text	Actual	Predicted	Possible Reason
tried	Positive	Negative	Ambiguous wording or limited context
microsoft shuts controversial botnet necurs iz...	Neutral	Negative	Sarcasm or informal language

best stream months big appreciation sports peo...	Positive	Neutral	Mixed sentiment within the tweet
excellent find hold entire parties using zoom ...	Neutral	Positive	Short text with insufficient contextual information
Unfortunately vlad lost body thanks listening	Neutral	Negative	Context-dependent sentiment wording

3.13 Final Model Selection

The final model was selected automatically by comparing F1-score values. Logistic Regression achieved the highest F1-score of 0.7560, compared with 0.7067 for Multinomial Naive Bayes. Therefore, Logistic Regression was selected as the best-performing model for this implementation. The trained model demonstrates useful predictive performance while still leaving scope for improvement through richer linguistic representations and advanced NLP techniques.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Results Obtained

The Sentiment Analysis project was successfully implemented in Python using Pandas, NLTK, Scikit-learn, Matplotlib, Seaborn, and WordCloud. The complete workflow covered data ingestion, quality assessment, three-class filtering, text preprocessing, TF-IDF vectorization, supervised classification, model comparison, confusion-matrix analysis, WordCloud generation, and error analysis.

The final modelling dataset contained 57,606 usable text records. An 80:20 stratified split produced 46,084 training samples and 11,522 testing samples. Logistic Regression achieved the best overall performance with 75.73% accuracy and a 0.7560 weighted F1-score.

4.2 Key Findings

1. The original dataset contains 74,682 records and 4 columns.
2. 686 missing Text values and 2,700 duplicate records were identified during inspection.
3. The original dataset contains Negative, Positive, Neutral, and Irrelevant classes.
4. After filtering to the required three classes, 59,119 records remained.
5. After text preprocessing, 57,606 usable text samples remained.
6. TF-IDF produced 5,000 numerical features using unigrams and bigrams.
7. The train-test split contained 46,084 training samples and 11,522 testing samples.
8. Multinomial Naive Bayes achieved 71.16% accuracy and 0.7067 F1-score.
9. Logistic Regression achieved 75.73% accuracy and 0.7560 F1-score.
10. Logistic Regression was selected as the best-performing model.
11. The test set contained 2,796 Logistic Regression misclassifications, showing that context-dependent language remains challenging.

4.2 Performance Summary

Model	Accuracy	Precision	Recall	F1-Score
Multinomial Naive Bayes	71.16%	71.45%	71.16%	70.67%
Logistic Regression	75.73%	75.68%	75.73%	75.60%

4.3 Conclusion

The Level 1 – Task 4 Sentiment Analysis project successfully transformed raw Twitter text into a machine-learning-ready representation and developed two supervised sentiment classifiers. The implementation demonstrated a complete workflow from data-quality assessment and text preprocessing to TF-IDF feature extraction, model training, evaluation, visualization, and error analysis.

The results clearly show that Logistic Regression performed better than Multinomial Naive Bayes on the prepared dataset, achieving 75.73% accuracy and a 0.7560 F1-score. The confusion matrix and classification report indicate stronger recognition of all three sentiment categories, although Neutral remains comparatively difficult because neutral expressions frequently depend on context rather than strongly polarized words.

Overall, the project demonstrates the practical value of machine-learning based sentiment analysis for applications such as customer feedback monitoring, social-media analytics, brand perception analysis, product-review analysis, and opinion tracking. At the same time, the error analysis shows that advanced language understanding is required to handle sarcasm, ambiguity, mixed sentiment, and context-dependent expressions more effectively.

4.4 Future Enhancements

1. Apply lemmatization or stemming and compare the effect on classification performance.
2. Experiment with word embeddings such as Word2Vec, GloVe, or contextual embeddings.
3. Evaluate transformer-based models such as BERT for improved contextual understanding.
4. Use hyperparameter tuning and cross-validation to optimize the classifiers.
5. Address class imbalance through class weighting or resampling where appropriate.
6. Expand error analysis using a larger manually reviewed sample of misclassified tweets.
7. Develop an interactive dashboard for real-time sentiment monitoring and trend analysis.
8. Add domain-specific sentiment lexicons and sarcasm-aware features for improved contextual interpretation.

4.5 References

1. Project-provided twitter_training.csv dataset and Level 1 – Task 4 implementation notebook.
2. Pandas library functionality used for dataset loading, filtering, cleaning, and tabular analysis.
3. NLTK functionality used for tokenization and English stopword processing.

4. Scikit-learn TfidfVectorizer, MultinomialNB, LogisticRegression, train_test_split, and evaluation metrics used in the implementation.
5. Matplotlib and Seaborn visualization libraries used for charts and confusion matrices.
6. WordCloud library used for sentiment-specific vocabulary visualization.
7. Level 1 – Task 4: Sentiment Analysis project requirements provided by OASIS INFOBYTE.

APPENDIX A: SCREENSHOT EVIDENCE

The following screenshots provide visual evidence for the major implementation stages documented in Chapter 3. Each figure corresponds to an output generated from the submitted Jupyter Notebook.

01_Dataset_Loading.png — Dataset loading and initial records

02_Sentiment_Distribution.png — Three-class sentiment distribution

03_Text_Preprocessing.png — Raw versus cleaned text example

04_TFIDF_Features.png — TF-IDF feature matrix

05_Train_Test_Split.png — Stratified training/testing split

06_Naive_Bayes.png — Multinomial Naive Bayes evaluation

07_Logistic_Regression.png — Logistic Regression evaluation

08_Model_Evaluation.png — Model performance comparison

09_Confusion_Matrix.png — Confusion matrices

10_WordCloud_Positive.png — Positive sentiment WordCloud

11_WordCloud_Neutral.png — Neutral sentiment WordCloud

12_WordCloud_Negative.png — Negative sentiment WordCloud

13_Error_Analysis.png — Representative error-analysis examples

APPENDIX B: OUTPUT FILES

The notebook generates the following analytical outputs in the output folder:

1. model_comparison.csv – comparison of Naive Bayes and Logistic Regression metrics.
2. classification_report_nb.csv – detailed classification report for Multinomial Naive Bayes.
3. classification_report_lr.csv – detailed classification report for Logistic Regression.
4. misclassified_examples.csv – representative Logistic Regression classification errors and possible reasons.

The main implementation notebook is Level1_Task4_Sentiment_Analysis.ipynb, and the screenshot evidence is stored in the screenshots folder using the numbered filenames referenced throughout this report.